

Pulso — Predice, nutre, entrena y protege tu corazón

Informe de seguridad: antes y después

Implementación de los controles de la matriz de activos
del Proyecto Integrador

Agustín Peralta Guarín

Natalia Moncaleano Montero

Detección de Intrusos y Técnicas de Ataque

Docente: José Eduardo Patiño Santafé

Universidad de San Buenaventura

Facultad de Ingeniería (Bogotá) — Ingeniería de Sistemas

Bogotá D.C., 1 de octubre de 2026

1. RESUMEN

Pulso es una PWA de salud cardiovascular (Next.js y PostgreSQL) desplegada con Coolify. La matriz de activos del Proyecto Integrador ([Plantilla_PI_Pulso.xlsx](#)) identificó diez activos con su vulnerabilidad, su amenaza, el riesgo de OWASP Mobile Top 10 y el control de ISO/IEC 27002:2022 que corresponde. Este informe documenta cómo estaba cada activo **antes**, qué se implementó y cómo quedó **después**, con la prueba que lo respalda.

El problema de fondo era la identidad: cada usuario era un UUID generado en el navegador y guardado en `localStorage`, que el servidor aceptaba sin verificar. Conocer el UUID de alguien equivalía a ser esa persona; y para editar o borrar ni siquiera hacía falta, porque bastaba el identificador de la fila. Hoy la sesión la emite el servidor y es la base de datos la que impide que un usuario vea los datos de otro.

Tabla 1. *Indicadores antes y después*

Aspecto	Antes	Después
Identidad del usuario	UUID en <code>localStorage</code> , legible por cualquier script, permanente	Cookie <code>__Host-</code> httpOnly, Secure, SameSite=Strict; la base guarda solo su hash; rota cada día
Server actions explotables	27 de 27 (21 aceptaban el <code>uid</code> del cliente, 6 operaban solo por <code>id</code>)	0: ninguna recibe el <code>uid</code> ; editar y borrar exigen <code>id</code> y dueño
Rutas de API con autenticación	0 de 11	11 de 11, más 8 rutas nuevas de sesión, supresión y bloqueo
Aislamiento en la base	Solo un filtro en el código (y ausente al editar o borrar)	RLS forzado en 11 tablas; sin sesión no se ve ninguna fila
Usuario de la app en Postgres	<code>pulso</code> , superusuario	<code>pulso_app</code> : sin DDL, sujeto a RLS
Auditoría	Ninguna	Tabla append-only; ni el dueño de la base la puede modificar
Límites de tasa	Ninguno	Por sesión y por IP en Postgres
Consentimiento (Ley 1581)	No se pedía	Explícito, versionado y revocable
Bloqueo del teléfono	No existía	Passkey opcional (huella, Face ID o PIN), aplicada en el servidor
Red Docker de la base	Compartida con 31 contenedores	Red propia (<code>pulso</code>); los vecinos no la alcanzan
Tráfico app → base	Sin cifrar	TLS 1.3 con certificado verificado (<code>verify-full</code>)
Tráfico app → Claude	Proxy interno en HTTP plano	API de Anthropic por HTTPS
Claves escritas en la imagen Docker	3 (Anthropic, Gemini y token del proxy)	0
Headers de seguridad	Ninguno	CSP con nonce, HSTS, <code>frame-ancestors</code> , nosniff y otros
Vulnerabilidades en dependencias	24 (1 crítica, 14 altas)	10 (0 críticas, 2 altas)

Aspecto	Antes	Después
Pruebas de seguridad	0	36 automatizadas (20 de base de datos, 16 HTTP)

Nota. Las cifras del “antes” salen del código del commit 93388a5 y del estado de producción medido antes del primer cambio; las del “después”, de producción al cierre de este informe.

2. ALCANCE Y MÉTODO

- **Base:** las diez filas de la matriz de activos, más todo hallazgo nuevo que apareció al revisar el código, la base de datos y la infraestructura (sección 5).
- **Revisión:** lectura completa del código de datos, de las 11 rutas de API y de los componentes cliente; inspección de la base y de la configuración del servidor (Coolify, Traefik, red Docker, imagen de la app).
- **Cambios:** en ramas separadas, probados primero contra una base PostgreSQL 17 descartable y un build de producción local, y recién después aplicados en producción, siempre con un respaldo previo de la base.
- **Verificación:** pruebas automatizadas que atacan cada control (sección 8), recorridos en el navegador y comprobaciones directas en el servidor.

3. ARQUITECTURA ANTES Y DESPUÉS

Figura 1. *Antes: identidad en el navegador y red compartida*

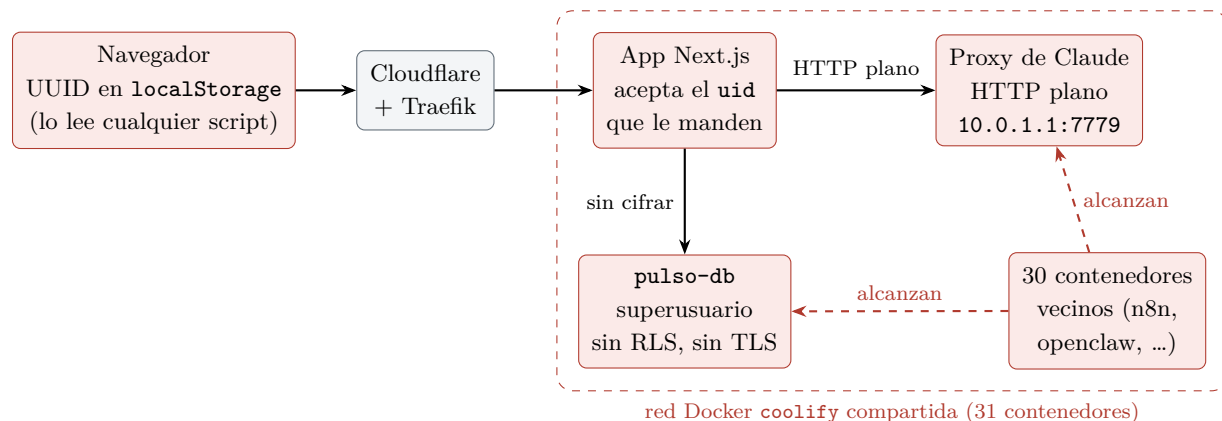
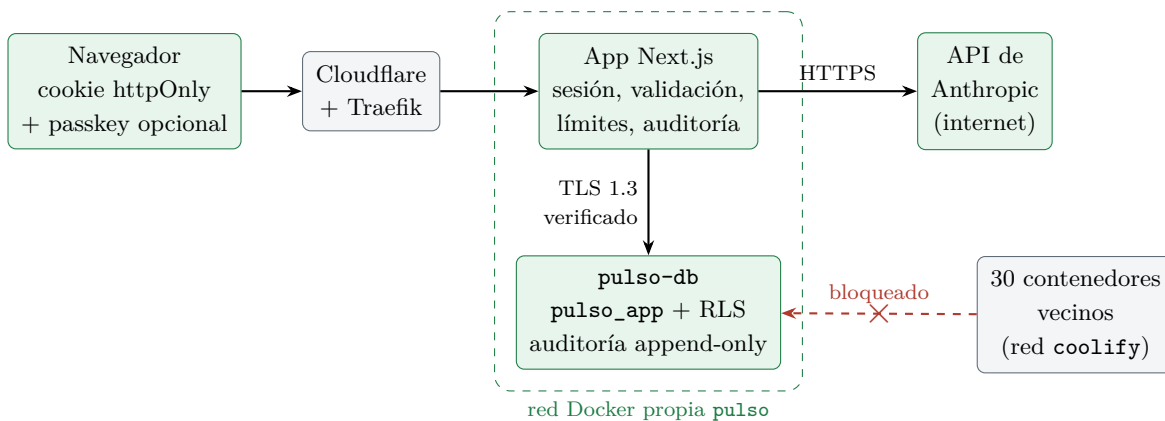


Figura 2. Después: sesión emitida por el servidor, red propia y tráfico cifrado

Nota. Traefik está conectado a las dos redes porque tiene que rutear las peticiones hacia la app. Los vecinos siguen en la red coolify, que ya no incluye a la base ni a la app.

4. LA MATRIZ, FILA POR FILA

Cada bloque reproduce la vulnerabilidad registrada en la matriz (*antes*), el control implementado (*después*) y la prueba que lo demuestra.

Fila 1 · Métricas de salud (Dashboard)

OWASP Mobile Top 10: M3 — Autenticación/autorización insegura

ISO/IEC 27002:2022: 8.3 Restricción del acceso a la información

ANTES

El server action recibía el `uid` desde el cliente y no verificaba la propiedad: quien obtuviera un UUID leía y modificaba las métricas de esa persona (STRIDE: *Information Disclosure, Tampering*).

DESPUÉS

Ninguna acción recibe el `uid`: lo saca de la cookie de sesión. Cada operación corre en una transacción con `app.uid` fijado y la base aplica RLS: aunque el código olvidara el filtro, no devolvería filas ajenas.

Cómo se verificó. prueba de base “RLS: cada usuario ve y toca solo lo suyo” (un usuario intenta leer, editar, borrar e insertar a nombre de otro: 0 filas o error de política); prueba HTTP “sin sesión, todas las rutas protegidas dan 401”.

Fila 2 · Asistente de recetas y mercado

OWASP Mobile Top 10: M4 — Validación insuficiente de entradas/salidas

ISO/IEC 27002:2022: 8.28 Codificación segura

ANTES

Los ingredientes iban al prompt sin sanitizar, `/api/recetas` no exigía autenticación ni validaba la longitud, y el historial del chat aceptaba cualquier rol (incluido `system`), lo que permitía reescribir las reglas del asistente (*prompt injection*).

DESPUÉS

Sesión obligatoria; validación con `zod` de tipo y largo de todo lo que entra; el historial solo admite `user` y `assistant` (máximo 12 turnos); el texto del usuario va envuelto en `<datos_usuario>` y el prompt de sistema aclara que es dato, no instrucción. En rutinas, las respuestas son valores cerrados del cuestionario.

Cómo se verificó. prueba HTTP “rutina: solo acepta las opciones del cuestionario” (un texto de inyección en vez de una opción se rechaza con 400).

Fila 3 · Generador de rutinas (contexto de salud)

OWASP Mobile Top 10: M6 — Controles de privacidad inadecuados

ISO/IEC 27002:2022: 5.34 Privacidad y protección de datos personales

ANTES

El sueño, el estrés y el avance del usuario se enviaban a un proveedor externo de IA sin minimizar los datos ni informarlo (Ley 1581 de 2012: los datos de salud son sensibles).

DESPUÉS

Consentimiento explícito y versionado antes del primer envío de datos de salud, en el momento de uso, explicando qué se envía, a quién y para qué; revocable desde `/privacidad`. El servidor lee de la base solo lo que la rutina usa, en vez de confiar en lo que manda el cliente. Página de privacidad con derecho de supresión.

Cómo se verificó. prueba HTTP “las rutas con datos de salud exigen consentimiento” (403 hasta autorizar); recorrido en el navegador del diálogo de consentimiento (rechazar no envía nada; autorizar registra el evento en la auditoría).

Fila 4 · Calendario de hábitos y notificaciones

OWASP Mobile Top 10: M8 — Configuración de seguridad incompleta

ISO/IEC 27002:2022: 8.9 Gestión de la configuración

ANTES

`/api/notificaciones/enviar` no exigía autenticación y aceptaba título, cuerpo y URL de destino arbitrarios: cualquiera podía mandarle a cualquier usuario una notificación con un enlace externo (phishing por push). Además, uno se podía suscribir a las notificaciones de otro.

DESPUÉS

Solo te podés enviar a tus propios dispositivos; título y cuerpo acotados; la URL tiene que ser una ruta interna, validada en el servidor y otra vez en el *service worker*; límite de 20 envíos por hora; suscripción atada a la sesión (máximo 5 por usuario).

Cómo se verificó. prueba HTTP “push: URL externa, texto largo o campos de más se rechazan” (`https://phishing.example`, `//phishing.example` y `javascript:` dan 400) y “el límite por hora corta con 429”.

Fila 5 · Score de riesgo cardiovascular

OWASP Mobile Top 10: M8 — Configuración de seguridad incompleta

ISO/IEC 27002:2022: 8.15 Registro de eventos

ANTES

La app no registraba ningún evento: imposible investigar un incidente o rebatir un reclamo por una recomendación errónea (STRIDE: *Repudiation*).

DESPUÉS

Tabla *auditoria* append-only: la app inserta pero no lee ni modifica, y un trigger impide actualizar, borrar o truncar incluso al dueño de la base. Se registra cada escritura, cada llamada a la IA (con el hash del informe que vio el modelo), la sesión y todo acceso denegado. La IP se guarda recortada (`/24` o `/48`).

Cómo se verificó. prueba de base “auditoría: la app inserta pero no lee; nadie modifica”; prueba HTTP “los rechazos quedan en la auditoría”. En producción se verificó que la IP auditada es la del cliente y no la del borde de Cloudflare.

Fila 6 · Centro de tips y claves de IA

OWASP Mobile Top 10: M1 — Uso indebido de credenciales

ISO/IEC 27002:2022: 8.6 Gestión de capacidades

ANTES

Ninguno de los 7 endpoints de IA exigía autenticación ni aplicaba límite de tasa por usuario o por IP: se podía agotar la cuota y disparar el costo del proyecto (denegación de servicio económica).

DESPUÉS

Sesión obligatoria en todas las rutas; límites en PostgreSQL por sesión y por IP (texto: 40/h por sesión y 120/h por IP; imágenes: 8/h y 24/h; creación de sesiones: 30/h por IP). La IP real se toma de Cloudflare solo si el par es de verdad un borde de Cloudflare, para que no se pueda falsificar.

Cómo se verificó. prueba de base “límite de tasa: corta al pasar el máximo”; prueba HTTP “el límite por hora corta con 429”.

Fila 7 · Identidad y sesión

OWASP Mobile Top 10: M9 — Almacenamiento inseguro de datos

ISO/IEC 27002:2022: 5.17 Información de autenticación

ANTES

El identificador era permanente, sin rotación ni expiración, y se guardaba en `localStorage`, accesible por JavaScript: un XSS o un script de terceros robaba la cuenta. El UUID además aparecía en las URLs de las fotos (STRIDE: *Spoofing*).

DESPUÉS

Token aleatorio de 256 bits en cookie `__Host-httpOnly`, Secure, SameSite=Strict; la base guarda solo su SHA-256. **Rotación diaria:** el token viejo queda dos minutos de gracia y después no sirve ni se puede refrescar. Vence a los 180 días sin uso. Las fotos van en una carpeta derivada del uid y solo las ve su dueño. Los usuarios anteriores reclaman su historial una sola vez.

Cómo se verificó. pruebas de base de rotación, revocación y vencimiento; prueba HTTP “rotación: un token de más de un día se reemplaza y el viejo vence” y “uid legado: se reclama una vez”; en el navegador, `document.cookie` queda vacío.

Fila 8 · Infraestructura: VPS, Coolify y base

OWASP Mobile Top 10: M2 — Seguridad inadecuada de la cadena de suministro

ISO/IEC 27002:2022: 8.22 Segregación de redes

ANTES

Los secretos vivían en variables de entorno, el proxy de IA era alcanzable por cualquier contenedor de la misma red Docker, y un contenedor vecino comprometido llegaba a la base (movimiento lateral). La app se conectaba como superusuario.

DESPUÉS

App y base en una red Docker propia: los 30 contenedores vecinos ya no llegan a la base. Rol `pulso_app` sin privilegios. Pulso dejó de usar el proxy de Claude del servidor. Las claves dejaron de ser variables de `build` (estaban escritas en la imagen) y se borraron las imágenes viejas. Se quitaron dos dependencias sin uso y se actualizó Next.js.

Cómo se verificó. desde un contenedor en la red coolify, la base responde “AISLADA”; la imagen nueva tiene 0 claves en su configuración y 0 capas que las mencionen; `npm audit` bajó de 24 vulnerabilidades a 10, sin críticas; prueba de base “la app no es superusuario ni puede hacer DDL”.

Fila 9 · Cliente PWA en el dispositivo

OWASP Mobile Top 10: M9 — Almacenamiento inseguro de datos

ISO/IEC 27002:2022: 7.9 Seguridad de los activos fuera de las instalaciones

ANTES

La app no pedía ningún bloqueo ni segundo factor: quien abriera el navegador del teléfono entraba directo al historial de salud (STRIDE: *Information Disclosure*).

DESPUÉS

Bloqueo opcional con passkey (WebAuthn: huella, Face ID o PIN del equipo), aplicado en el servidor: sin desbloquear, la sesión no resuelve a ningún usuario y todas las rutas responden 423. Se bloquea solo a los 15 minutos sin uso y la app desmonta los datos de la pantalla. Bloqueada no se puede borrar los datos ni desactivar el bloqueo. Además, “Borrar mis datos” limpia base, fotos, cookie, almacenamiento local, caché y suscripción push.

Cómo se verificó. prueba HTTP con un autenticador WebAuthn de software y firmas reales: sin verificación de usuario se rechaza, una firma corrupta y una passkey ajena se rechazan, un desafío no se puede reusar y, bloqueada, todo da 423. Corrió también en producción. En el navegador, la pantalla de bloqueo no deja ningún dato en la página.

Fila 10 · Comunicación entre capas

OWASP Mobile Top 10: M5 — Comunicación insegura

ISO/IEC 27002:2022: 8.20 Seguridad de redes

ANTES

En modo *oauth* los prompts con datos de salud viajaban a un proxy interno sin TLS; el tráfico entre la app y la base iba sin cifrar; no había headers de seguridad (STRIDE: *Information Disclosure*).

DESPUÉS

Claude solo por HTTPS: con un proxy en HTTP plano, la app lo ignora y usa la API de Anthropic. TLS hacia PostgreSQL con verificación del certificado y del nombre del servidor contra la CA de Coolify. Headers: CSP con nonce por petición, HSTS, **frame-ancestors 'none'**, nosniff, Referrer-Policy, Permissions-Policy y COOP; **no-store** en la API.

Cómo se verificó. en producción, 1 de 1 conexiones de la app a la base con TLS 1.3; en local, con una CA ajena la app se niega a conectar; prueba de **provider** “el proxy tiene que cifrar el tráfico”; prueba HTTP “headers de seguridad” y “la página trae el nonce en sus scripts”.

5. HALLAZGOS FUERA DE LA MATRIZ

Al revisar el código y el servidor aparecieron problemas que la matriz no registraba. Todos quedaron corregidos.

Tabla 2. *Hallazgos adicionales*

Hallazgo	Antes	Después
Borrado y edición por id (IDOR)	Seis acciones operaban con el <code>id</code> de la fila solo: alcanzaba con ese identificador, sin conocer el UUID	Exigen <code>id</code> y dueño; el intento sobre una fila ajena queda auditado como denegado
Subida arbitraria de archivos	<code>uploadRecetaImagen</code> escribía en disco cualquier <i>data URL</i> que mandara el cliente	Se eliminó; la imagen la guarda el servidor, verificando los bytes del formato (PNG, JPEG o WebP)
Fuga de detalles internos	Los mensajes de error de PostgreSQL llegaban al cliente, y <code>/api/recetas/imagen</code> devolvía el <i>stack</i> completo	El cliente recibe un mensaje genérico; el detalle va solo al log
Caché pública de fotos	Las fotos se servían con <code>Cache-Control: public</code>	<code>private</code> , y solo al dueño
Claves dentro de la imagen Docker	Tres claves marcadas como variables de <i>build</i> quedaban escritas en la imagen	Solo de <i>runtime</i> ; imagen reconstruida e imágenes viejas borradas
Next.js con vulnerabilidad crítica	15.5.15, con una evasión de <i>middleware</i> entre sus fallas	15.5.27; además, la autenticación no depende del <i>middleware</i>

6. CONTROL DE ACCESO BASADO EN ROLES EN LA BASE

Tabla 3. *Roles de PostgreSQL*

Rol	Puede	No puede
Dueño (superusuario)	DDL, migraciones y respaldos	— (no lo usa la app)
<code>pulso_app</code>	Leer y escribir datos, solo de las filas de su <code>app.uid</code> ; insertar en la auditoría; ejecutar las funciones <code>pulso_*</code>	Crear o borrar tablas; leer sesiones, identidades o límites; leer o modificar la auditoría
<code>pulso_auditor</code>	Leer la auditoría	Todo lo demás

Nota. Las tablas de sesión no tienen permisos para la app: solo se tocan a través de funciones *security definer*. Así la app no puede leer sesiones ajenas ni crear una sesión para un usuario que elija: el identificador nuevo lo genera la base. RLS “cierra por defecto”: sin `app.uid`, la comparación da falso y no se ve ninguna fila.

7. EL CAMBIO EN EL CÓDIGO

Identidad

Antes — el identificador lo generaba y guardaba el navegador:

```
export function getAnonymousId(): string {
  let uid = localStorage.getItem(STORAGE_KEY);
  if (!uid) { uid = uuidv4(); localStorage.setItem(STORAGE_KEY, uid); }
  return uid;
}
```

Después — lo emite el servidor; la base guarda solo el hash:

```
const token = randomBytes(32).toString("base64url"); // 256 bits
await pool.query("select pulso_crear_sesion($1, $2) as uid",
  [hashToken(token), DIAS_SESION]); // solo el hash
(await cookies()).set("__Host-pulso_sesion", token,
  { httpOnly: true, secure: true, sameSite: "strict", path: "/" });
```

Borrar una receta

Antes — cualquier receta, de cualquier usuario, con su id:

```
export async function eliminarReceta(id: string) {
  await pool.query(`delete from recetas_guardadas where id = $1`, [id]);
}
```

Después — sesión, validación, dueño, RLS y auditoría:

```
export async function eliminarReceta(id: string) {
  return escritura("receta.eliminar", idSchema, id,
    async ({ db, uid, auditar }, id) => {
      const r = await db.query(
        `delete from recetas_guardadas where id = $1 and uid = $2`, [id, uid]);
      if (r.rowCount === 0) {
        await auditar({ accion: "receta.eliminar", recursoId: id,
          resultado: "denegado" });
        return ERROR_NO_ENCONTRADO;
      }
    })
  ...
}
```

La base como última barrera

```
create policy propietario on recetas_guardadas to pulso_app
  using (uid = current_setting('app.uid', true))
  with check (uid = current_setting('app.uid', true));
```

Notificaciones push

Antes — destinatario, texto y enlace los elegía quien llamara:

```
const { uid, title, body, url } = await req.json();
```

Después — solo a uno mismo, con texto acotado y URL interna:

```
export const POST = rutaProtegida(
  { nombre: "push.enviar", cuerpo: cuerpoPushSchema,
    limites: (uid) => [limite("push", uid)] },
  async ({ uid, cuerpo }) => {
    // uid: el de la sesion. cuerpo.url: validada como ruta interna.
  });
```

8. VERIFICACIÓN

Tabla 4. Pruebas automatizadas

Suite	Resultado	Qué demuestra
base.test.mjs	20 de 20	La app no es superusuario ni hace DDL; RLS aísla y cierra por defecto; un uid legado se reclama una vez; sesiones revocadas, vencidas o rotadas no sirven; auditoría inmutable; límite de tasa; supresión solo de lo propio; bloqueada no hay usuario ni borrado; desafíos de un solo uso
http.test.mjs	16 de 16	401 en las 11 rutas sin sesión; cookie inventada rechazada; Origin ajeno rechazado; consentimiento exigido; push malicioso rechazado; 429 al pasar el límite; IDOR de fotos da 404; rotación sin pisar la cookie; bloqueo con firmas WebAuthn reales
provider-fallback.mjs	pasa	El proxy de Claude se acepta solo con TLS
Pruebas previas del motor	70 de 70	Los cambios no rompieron el motor de predicción

Nota. Las suites de base y HTTP corren completas contra una base PostgreSQL 17 descartable con usuarios “legado” sembrados. Contra producción corren las pruebas que no necesitan preparar datos (10 de 16, todas en verde); la de bloqueo crea una identidad de prueba y la borra al terminar.

Además, en producción se comprobó directamente:

- la app se conecta como `pulso_app` (no superusuario), con TLS 1.3, desde la red `pulso`;
- desde la red `coolify`, la base no responde, y los demás sitios del servidor siguieron funcionando igual;
- la imagen de la app no contiene ninguna clave y no quedan imágenes anteriores;
- la IP registrada en la auditoría es la del cliente, recortada, y no la de Cloudflare;
- en el navegador: `document.cookie` vacío, ningún script sin nonce, ninguna violación de CSP, y la pantalla de bloqueo sin datos en la página.

9. RIESGOS RESIDUALES Y PENDIENTES

- **Crédito de los proveedores de IA.** La clave de Anthropic es válida pero la cuenta no tiene saldo, y la de Gemini tiene la cuota agotada: hasta cargar crédito, las funciones de IA muestran “el servicio de IA no respondió”. No es un problema de seguridad: el resto de la app funciona.

- **Inyección de prompt.** Se acota (roles filtrados, datos etiquetados, valores cerrados), pero ningún control la elimina del todo en un modelo de lenguaje. El impacto queda limitado al propio usuario: el modelo no tiene herramientas ni acceso a datos de otros.
- **El bloqueo es opcional.** Protege a quien lo activa; hacerlo obligatorio exigiría que todo usuario tenga un teléfono con passkeys.
- **El proxy de Claude del servidor** sigue alcanzable por cualquier contenedor, con su token como único límite. Ya no afecta a Pulso, que no lo usa, pero sí al servidor.
- **Dependencias con arreglo mayor pendiente:** `postcss` dentro de Next.js (solo en el build) y `jsdiffpatch` en el SDK de IA requieren Next.js 16 y AI SDK 7. Ninguna se ejecuta con entrada del usuario.
- **Rotación de claves:** las que estuvieron escritas en la imagen nunca salieron del servidor (no hay registro de imágenes), pero conviene rotarlas en Anthropic y Google.
- **El dueño de la base** conserva acceso total; se audita lo que hace la app, no lo que haga un operador con el superusuario.

ANEXO: TRAZABILIDAD

- **Repositorio:** <https://github.com/Agustin1231/pulso>. Commits: `0e8f62a` (sesión del servidor, RLS y controles), `494c6d1` y `6c307db` (documentación y red dedicada), `9c147d6` (rotación, bloqueo con passkey y TLS).
- **Documentación técnica:** `docs/seguridad.md`; migraciones `db/schema.sql` y `db/seguridad.sql`.
- **RespalDOS de la base** previos a cada cambio, en el servidor: `/root/backups/pulso-db-20261001T153953Z.dump`, `...T163408Z.dump` y `...T173519Z.dump`.
- **Base normativa:** OWASP Mobile Top 10 (2024); ISO/IEC 27002:2022; Ley 1581 de 2012 (protección de datos personales, art. 5 datos sensibles, art. 6 autorización, art. 8 derechos del titular).